温州肯恩大学
WENZHOU-KEAN UNIVERSITY

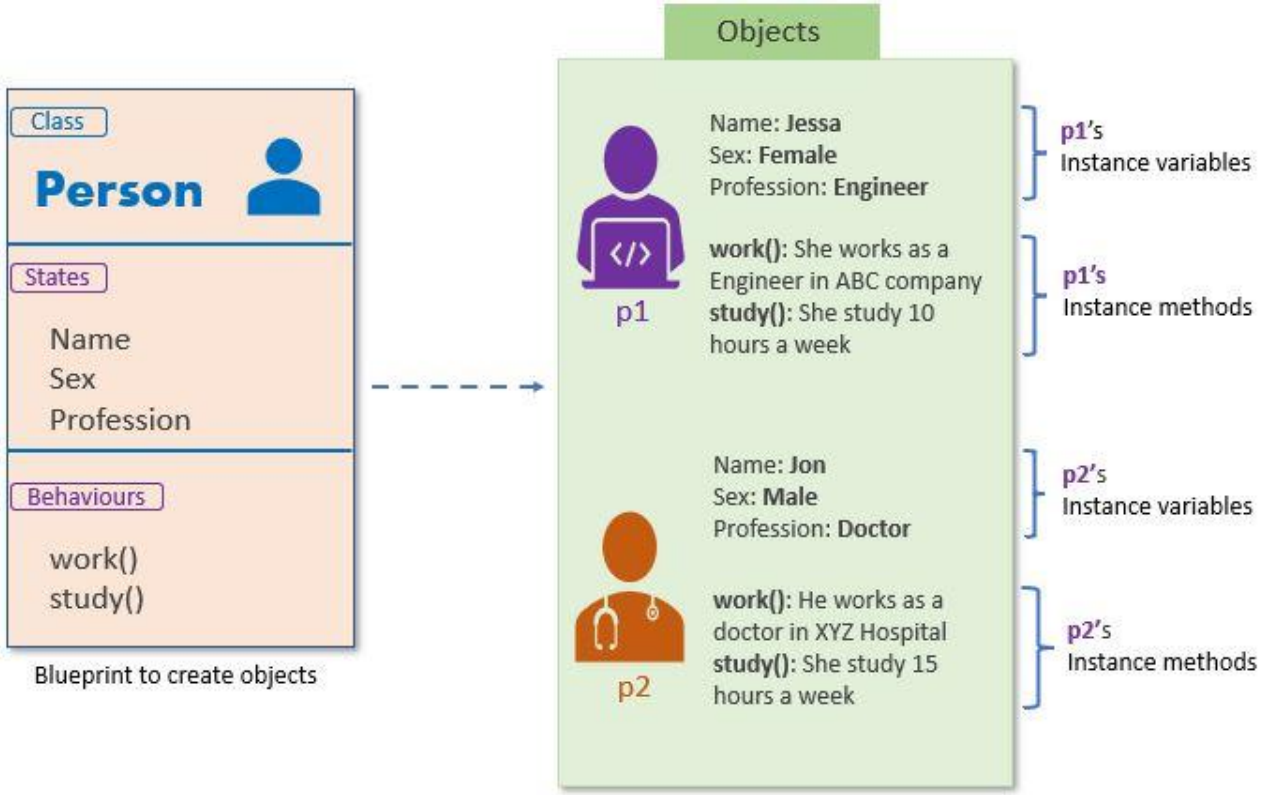
MGS 3101 W03 Foundations of Business Analytics

NumPy and Pandas

Dawei Chen

Week 2-2, 11 Sep 2025

Python Basics – Classes & Objects



https://pynative.com/python-classes-and-objects/

- The class is a user-defined data structure that binds the data members and methods into a single unit.
- An object is an instance of a class.
- Everything is an object.
- Each object has its own identity, attributes (state) and methods (behavior).

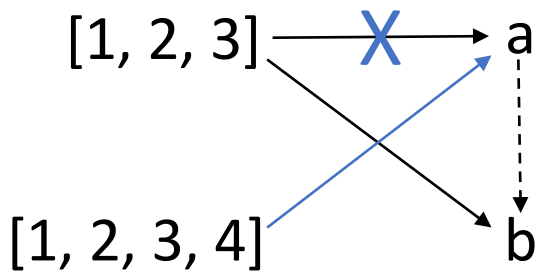
Revisit – Assignment Operator

append() method in Python is used to add a single item to the end of list. This method modifies the original list and does not return a new list

Case 1:

```
a = [1, 2, 3]
b = a
a = [1, 2, 3, 4]
```

What is a? b?

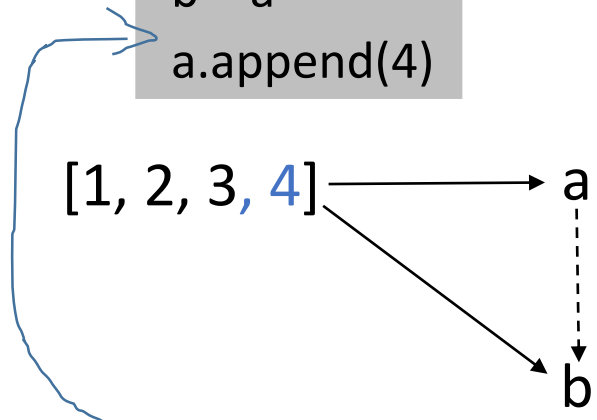


a = [1, 2, 3, 4]
b = [1, 2, 3]

Case 2:

```
a = [1, 2, 3]
b = a
a.append(4)
```

What is a? b?



a = [1, 2, 3, 4]
b = [1, 2, 3, 4]

```
c = b
d = c
```

What is c? d?

c = [1, 2, 3, 4]
d = [1, 2, 3, 4]

Key Takeaways

- Assignment (=) binds a name to an object; it is not equality.
- == checks value equality; is checks object identity.
- b = a makes b refer to the same object as a (no copy). What if b = a.copy()?
- Reassigning a name points it to a new object; mutating a shared mutable object (list/dict/set) affects all names.
- When creating or reassigning variables, pause to confirm which object a name references—especially in larger projects with many similar variables.

Python Basics – File Paths

- Definition: A file path is the **address** of a file or folder.
  - It describes the route through the file system, from a starting point (the root or current working directory) through folders, to the target item
- Type
  - Absolute path: starts at the file system root
    - Windows: C:/Users/you/project/data/train.csv
  - Relative path: starts at the current working directory
    - Example: data/train.csv
- In Jupyter
  - Where am I? %pwd
- My habit
  - ProjectName/ data, notebooks, results

Where is 'train.csv'?

```
data = pd.read_csv('train.csv')
```

```
data = pd.read_csv('./train.csv')
```

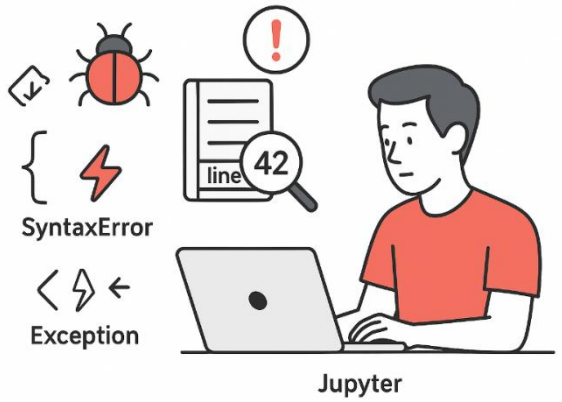
```
data = pd.read_csv('../train.csv')
```

```
data = pd.read_csv('../../train.csv')
```

```
fig.savefig('results/my_plot.pdf')
```

Python Basics – Errors and Exceptions

- What can go wrong
  - **SyntaxError:** Python can't parse your code (typo/format) Chinese characters
  - **Exceptions (runtime):** Code parsed, but fails while running
- Read the traceback
  - Last line = exception type + message
  - Use file + line number, start fixing at the highest frame in your code
  - Get help from AI tools, online community (e.g., Stack Overflow), Google, etc.
- Common exceptions in analysis
  - **NameError:** variable not defined
  - **TypeError:** wrong type /wrong argument
  - **ValueError:** right type, bad value
  - **KeyError:** missing column/key
  - etc.



Generated by ChatGPT5

Revisit DieTanic
Understand Python Code

NumPy (Numerical Python) – Quick Intro

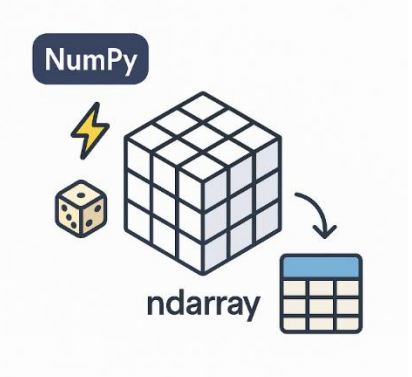
```
[1]: import numpy as np

[2]: data = np.array([[1.5, -0.1, 3], [0, -3, 6.5]])
     data

[2]: array([[ 1.5, -0.1,  3. ],
            [ 0. , -3. ,  6.5]])
```

- What it is
  - Core library for **numerical computing** in Python
- Core objects
  - ndarray (NumPy array): typed, fixed-size, multi-dimensional
- Why it's useful
  - Fast vectorized math (C-backed)
  - Memory-efficient for **large data**
  - Random number generation (numpy.random)
- NumPy ⟷ Pandas
  - Pandas Series/DataFrame are built on top of NumPy
  - Most DataFrame ops delegate to ndarry under the hood

*Direct NumPy use is **optional** for **beginner projects.***



Generated by ChatGPT5

Pandas – Three Main Objects

- Series
  - A one-dimensional array-like object containing a sequence of **values** (of similar types to NumPy types) of the same type and an associated array of data labels, called its **index**.
- DataFrame *Think: a spreadsheet*
  - A rectangular **table** of data and contains an **ordered**, **named** collection of **columns**, each of which can be a different value type (numeric, string, Boolean, etc.).
  - Both row and column are indexed.
- Index
  - Immutable sequence of axis labels
  - Powers alignment, selection, and joins
  - Stores axis name/metadata

```
[1]: import pandas as pd

[2]: data = pd.Series([4.8, 3.5, 4.5, 3.2],
                      index=["A", "B", "C", "D"])
     data

[2]: A    4.8
     B    3.5
     C    4.5
     D    3.2
     dtype: float64

[3]: dataDF = {"student": ["A", "B", "C", "D"],
               "year": [3, 4, 4, 3],
               "gpa": [4.8, 3.5, 4.5, 3.2]}
     df = pd.DataFrame(dataDF)
     df
```

[3]:

	student	year	gpa
0	A	3	4.8
1	B	4	3.5
2	C	4	4.5
3	D	3	3.2

```
[4]: df.index

[4]: RangeIndex(start=0, stop=4, step=1)

[5]: data.index

[5]: Index(['A', 'B', 'C', 'D'], dtype='object')
```

Pandas – Essential Functionality (Important!)

- Reindexing
- Dropping Entries from an Axis
- Indexing, Selection, and Filtering
- Arithmetic and Data Alignment
- Function Application and Mapping
- Sorting and Ranking
- Axis Indexes with Duplicate Labels

(Wes, Ch5)

How we will learn

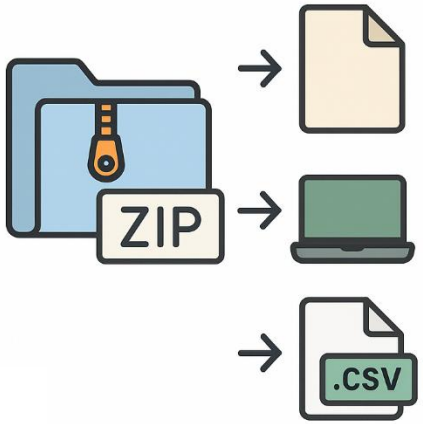
- Step-by-step with a real dataset
- Hands-on exercise in class

Attributes and Methods
of DataFrames

Mini Task – Load & Prep Dataset

1) Download **ml-100k.zip** from Canvas (week 2)
2) Unzip and read the **README** file (see description of **u.data**)
3) Place **u.data** in your preferred folder (or upload via Jupyter file browser)
4) In Jupyter Notebook
   - pd.read_csv() to load **u.data** *what happen?*
   - Set proper column index/names
   - Save as a **.csv** file

   which function?



Please also try to read the **u.item** file